

Big(O)

TIP101

Agenda

1 Welcome 🙌

2 Big(O)

3 **BREAK**

4 Collaborative Problem Solving

5 Wrap Up

10 minutes

35 minutes

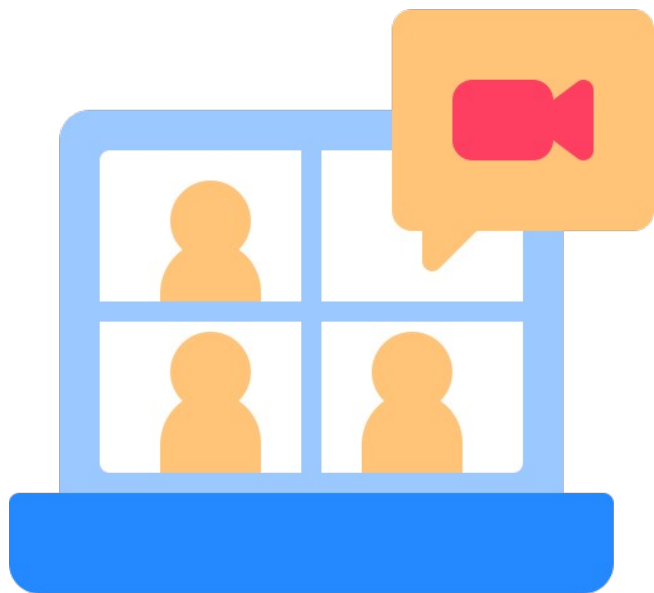
5 minutes

60 minutes

10 minutes

10 minutes

Welcome! 🖐️



**Turn on your
cameras**



**Fill out the entry
ticket**



Icebreaker

5 minutes

Question:

Let's say CodePath takes a visit to your city, what is one thing that we have to do while there?

Answer:

Share your answer in the Zoom chat below. Multiple options are encouraged.

Your Feedback



Glows

- + Students glad to have learned the rest of UMPIRE
- + Students feel like practice material is high quality.



Grows

- Students want more instructor demo time.
- Communication is confusing with slack + emails.

35 minutes

Big(O)

What is Big(O)?

When someone asks you "**what is the Big(O)**", they want to know . . .

- * How fast does your code run? (**time complexity**)
- * How much memory does your code take? (**space complexity**)
- * This is super important in industry ("How fast can we process a user request?"..."How much storage should we pay for per month?")

BIG O NOTATION

- $O(1) = O(\text{yeah})$
- $O(\log n) = O(\text{nice})$
- $O(n) = O(\text{ok})$
- $O(n^2) = O(\text{my})$
- $O(2^n) = O(\text{no})$
- $O(n!) = O(\text{mg})$

What is Big(O)?

When someone asks "What is the **Big(O)**", they want to know:

- * How fast does your algorithm run? (**time complexity**)
- * How much memory does it take? (**space complexity**)
- * This is super important! ("How fast can we process a user request?"..."How much storage should we pay for per month?")

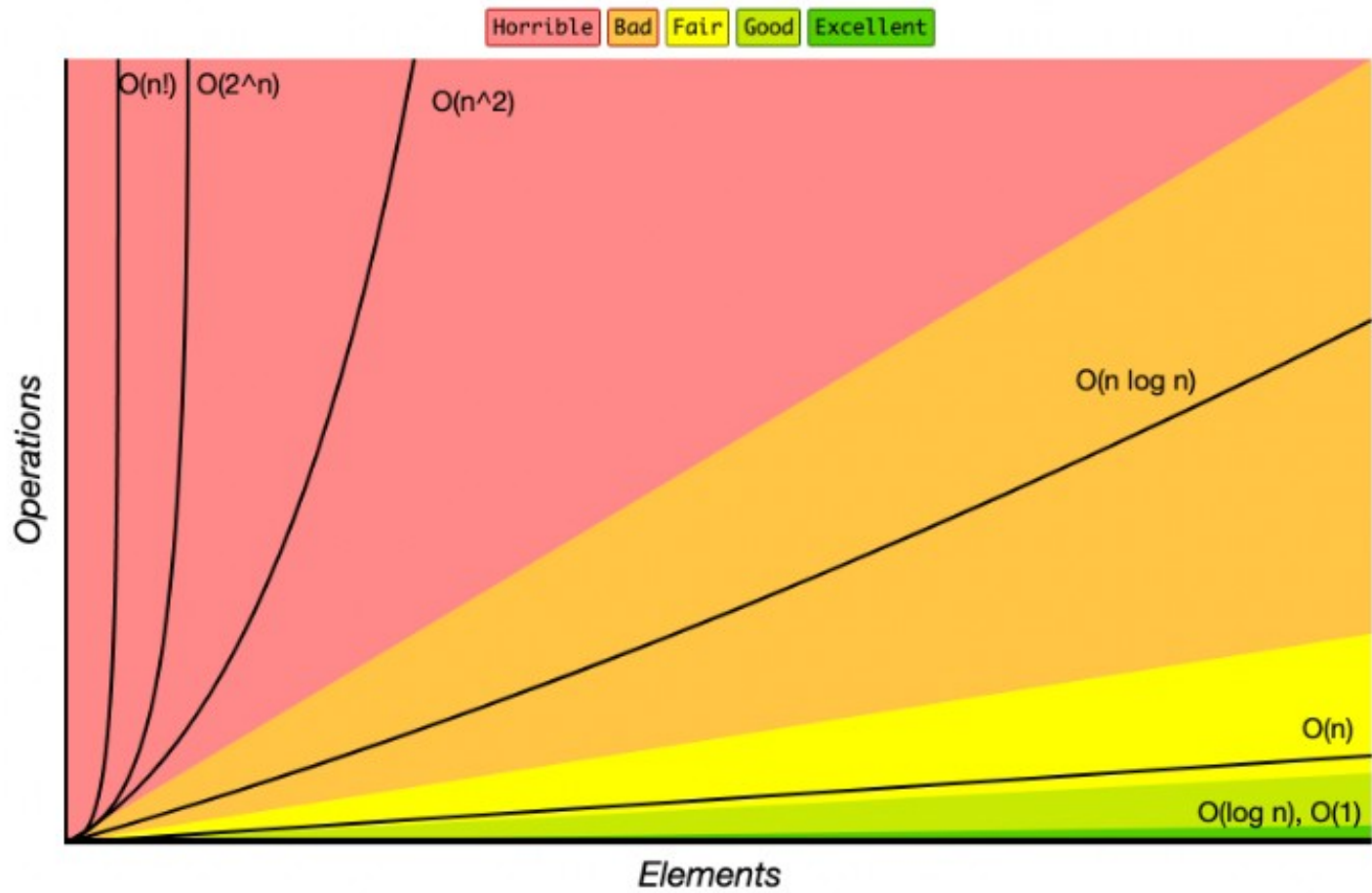
Note:

Big(O) is related to how speed and space increase **as your input size increases**.

For example: A list of 5 elements vs list of 10 elements.

- $O(1) = O(\text{yeah})$
- $O(\log n) = O(\text{nice})$
- $O(n) = O(\text{ok})$
- $O(n^2) = O(\text{my})$
- $O(2^n) = O(\text{no})$
- $O(n!) = O(\text{mg})$

BIG O NOTATION



Common Complexities (Fastest to Slowest)

Function	Time	Space
Constant	$O(1)$ - Constant time	$O(1)$ - Constant space
Logarithmic	$O(\log(n))$ - Logarithmic time	$O(\log(n))$ - Logarithmic space
Linear	$O(n)$ - Linear time	$O(n)$ - Linear space
Log-linear	$O(n \log(n))$ - Log-linear time	$O(n \log(n))$ - Log-linear space
Quadratic	$O(n^2)$ - Quadratic time	$O(n^2)$ - Quadratic space

Common Complexities (Fastest to Slowest)

Function	Big(O)	Examples
Constant	$O(1)$	odd or even number
Logarithmic	$O(\log(n))$	finding element on sorted array with binary search
Linear	$O(n)$	find max element in unsorted array, typical for loop
Log-linear	$O(n \log(n))$	sorting elements in array with merge sort
Quadratic	$O(n^2)$	sorting array with bubble sort, for loop within another

Common Complexities (Fastest to Slowest)

Function	Time	Space
Constant	$O(1)$ - Constant time	$O(1)$ - Constant space
Logarithmic	$O(\log(n))$ - Logarithmic time	$O(\log(n))$ - Logarithmic space
Linear	$O(n)$ - Linear time	$O(n)$ - Linear space
Log-linear	$O(n \log(n))$ - Log-linear time	$O(n \log(n))$ - Log-linear space
Quadratic	$O(n^2)$ - Quadratic time	$O(n^2)$ - Quadratic space

Common Complexities (Fastest to Slowest)

Function	Time	Space
Constant	$O(1)$ - Constant time	$O(1)$ - Constant space
Logarithmic	$O(\log(n))$ - Logarithmic time	$O(\log(n))$ - Logarithmic space
Linear	$O(n)$ - Linear time	$O(n)$ - Linear space
Log-linear	$O(n \log(n))$ - Log-linear time	$O(n \log(n))$ - Log-linear space
Quadratic	$O(n^2)$ - Quadratic time	$O(n^2)$ - Quadratic space



Try It!

5 minutes

What is the Big(O)?

```
def get_element_at_index(lst, index):  
    if index < 0 or index >= len(lst):  
        return None # Handle out-of-bounds cases  
    return lst[index]
```

What is the Big(O)?

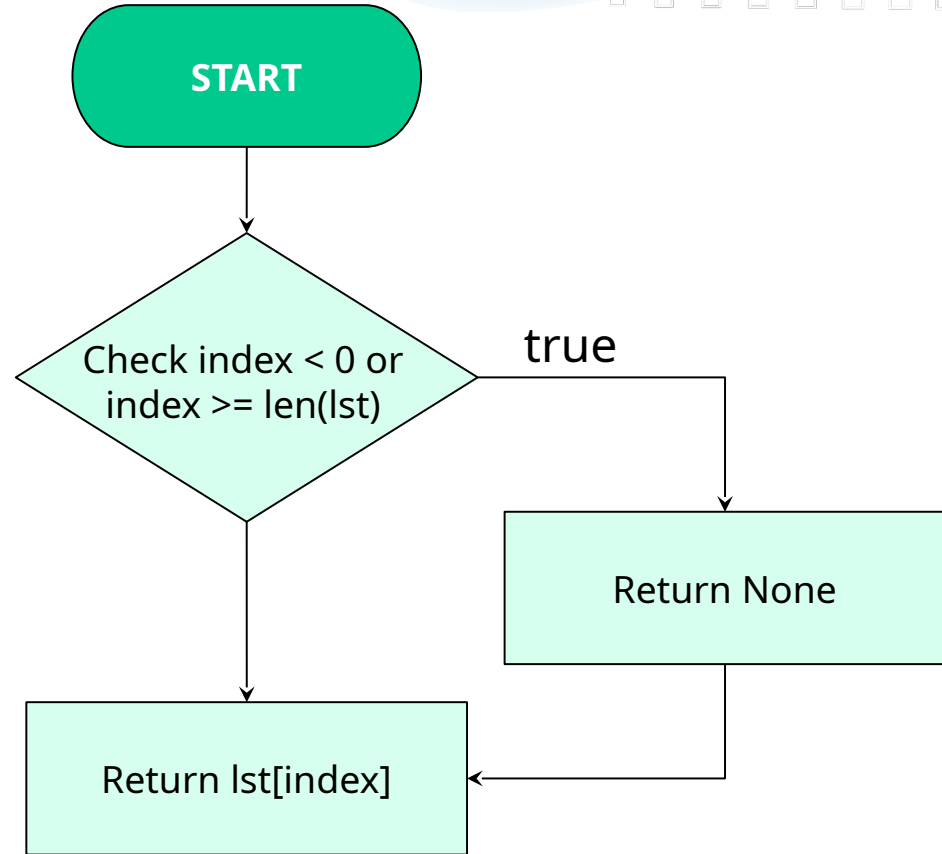
```
def get_element_at_index(lst, index):  
    if index < 0 or index >= len(lst):  
        return None # Handle out-of-bounds cases  
    return lst[index]
```


Time Complexity: get_element_at_index

lst = [a, b, c, d, e]

index = 2

access element at index 2
O(1) constant time



Space Complexity: `get_element_at_index` $O(1)$

Input List: [a, b, c, d, e]

Variables: 1st, index (constant space)

Output: single element or None (constant space)

Q: "How might the space complexity change if our input list grows in size?"

The memory usage is **constant**, $O(1)$, because it **doesn't grow** with the size of the input list. This makes the function memory-efficient, even for very large inputs.



Try It!

5 minutes

What is the Big(O)?

```
def sum_array(arr):  
    total = 0  
    for num in arr:  
        total += num  
    return total
```

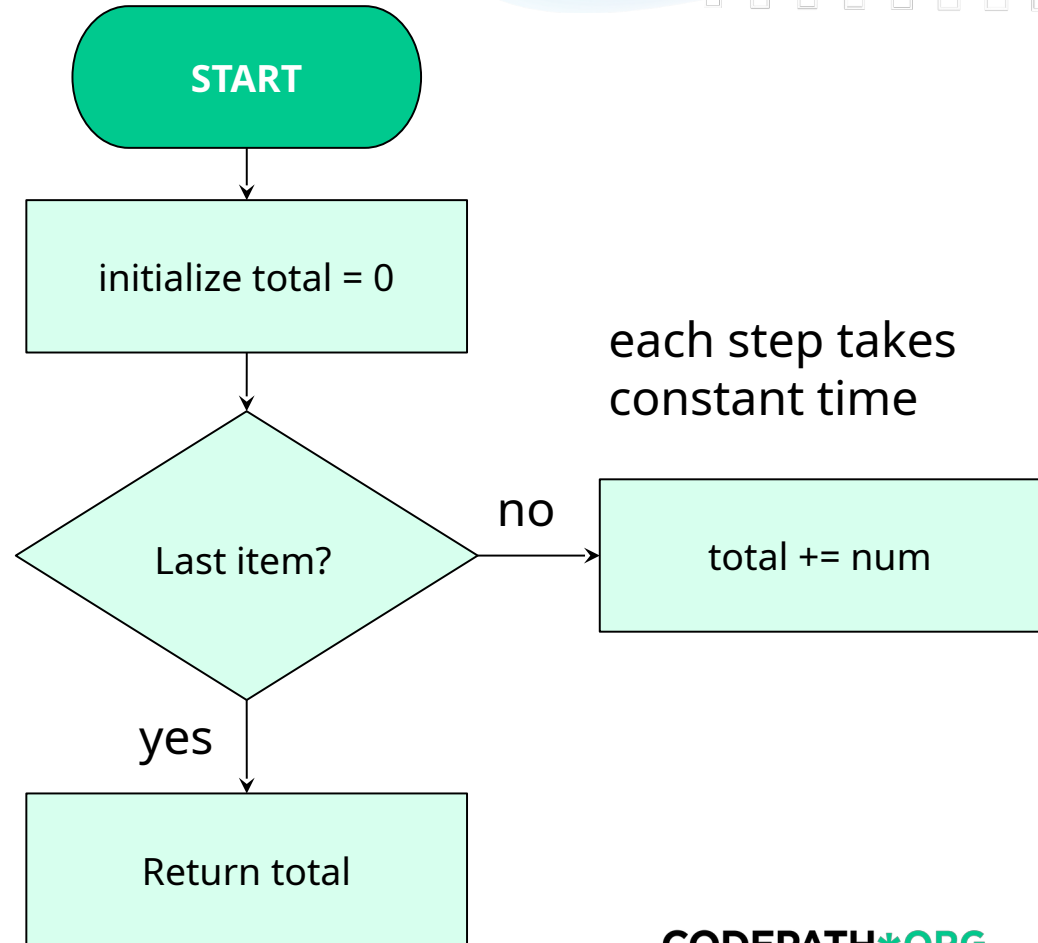
What is the Big(O)?

```
def sum_array(arr):  
    total = 0  
    for num in arr:  
        total += num  
    return total
```

Time Complexity: sum_array

`arr = [a, b, c, d, e]`

loop runs ***n*** times



Space Complexity: sum_array

Input List: [a, b, c, d, e]

Variables: total, num (constant space)

Output: total (constant space)

Space Complexity: sum_array

Input List: [a, b, c, d, e]

Variables: total, num (constant space)

Output: total (constant space)

The input list, [a, b, c, d, e], is provided as input, so the function **doesn't create** any new data structures that depend on its size.

The output is also a single value, total, which **doesn't grow** with the size of the array. Thus, the space complexity is $O(1)$, making the function memory efficient.



Try It!

5 minutes

What is the Big(O)?

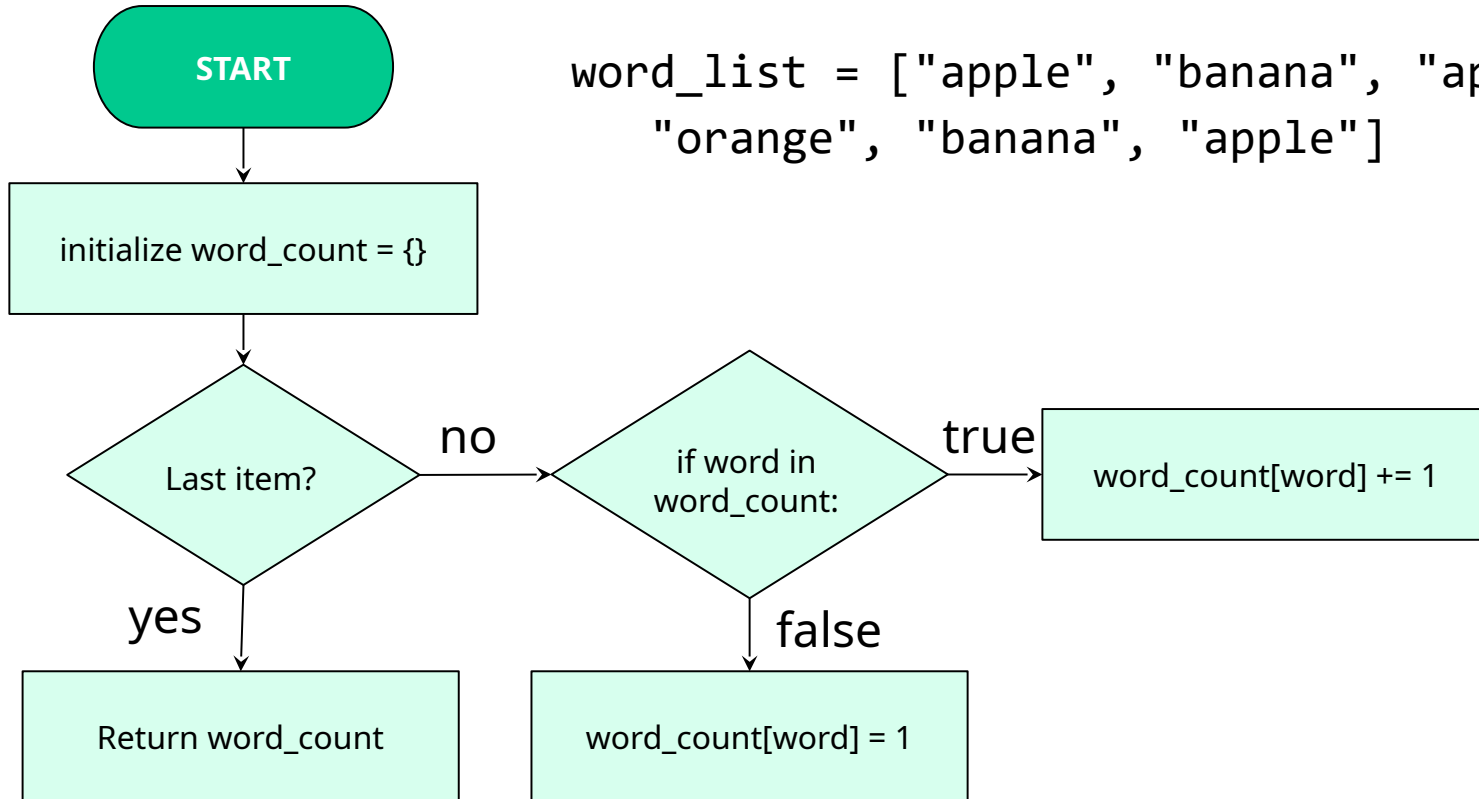
```
def count_word_frequencies(word_list):  
    word_count = {}  
    for word in word_list:  
        if word in word_count:  
            word_count[word] += 1  
        else:  
            word_count[word] = 1  
    return word_count
```


What is the Big(O)?

```
def count_word_frequencies(word_list):  
    word_count = {}  
    for word in word_list:  
        if word in word_count:  
            word_count[word] += 1  
        else:  
            word_count[word] = 1  
    return word_count
```

Time Complexity: count_word_frequencies

```
word_list = ["apple", "banana", "apple",  
             "orange", "banana", "apple"]
```



Space Complexity: count_word_frequencies $O(n)$

Input List: ["apple", "banana", "apple", "orange", "banana", "apple"]

Dictionary:

apple	3	banana	2	orange	1
-------	---	--------	---	--------	---

Q: What is the worst case? All words being unique would result in the largest output size. If all words were unique, this output size would grow linearly: $O(n)$



Try It!

5 minutes

What is the Big(O)?

```
def sum_top_100(arr):  
    total = 0  
    for i in range(100):  
        total += arr[i]  
    return total
```

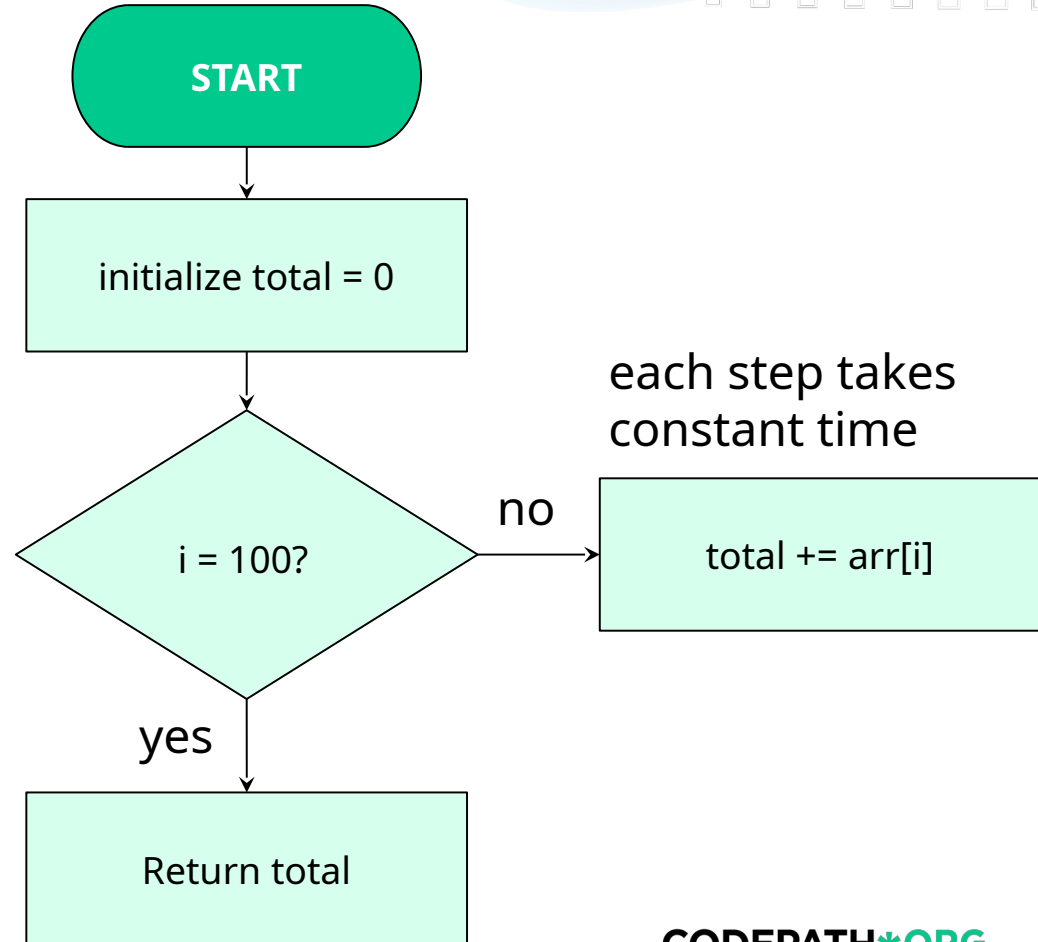
What is the Big(O)?

```
def sum_top_100(arr):  
    total = 0  
    for i in range(100):  
        total += arr[i]  
    return total
```

Time Complexity: sum_top_100

`arr = [a, b, c, d, e]`

loop runs **100** times



Space Complexity: sum_top_100

Input List: [a1, a2, a3, ..., a100, ...]

Variables: total, i (constant space)

Output: total (constant space)

Q: What is the worst case of our code in terms of space? What if we inputted an array with 10,000 elements?

A: Space would still be constant, as it **doesn't grow** with our input data.

Break Time!

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



60 minutes

Collaborative Problem Solving



Breakout Rooms

60 minutes

1. 🏃 Navigate to your breakout room.
2. 🙌 Introduce yourselves!
3. 🧑💻 Work on the problems for your selected unit.

10 minutes

Wrap Up



One Good Thing

Let's reflect on today's session. Maybe you got a question answered, overcame a specific coding challenge, or learned how to do something completely new!

In the chat, share at least one good thing you've learned or experienced during the session today.

Let's celebrate together!



Session Survey Time!





Final Reminders

- ☐ Finish the problem sets for additional practice
- ☐ Take the weekly HackerRank assessment by Sunday @ midnight
- ☐ Visit a **Study Hall Session** to deepen your understanding, work through challenges, and connect with course material alongside your peers!
- ☐ Next Session Lookahead:
 - **OOP** (Object Oriented Programming, how to define custom classes in Python)
 - **Linked Lists** (Will compare Linked Lists to normal lists, how to connect nodes and navigate through a linked list)
- ☐ **Buddy Breakout Group** forms are open! If you have a group you'd like to work with starting Week 5, check your email and [complete the form](#) to opt in!